CASE STUDY REPORT

# 13. Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Applied to the Vehicle Service Request Management System

**Submitted by:** SUHRITHA SRI
**Department:** Computer Science and Business Systems (CSBS)
**Register No.:** 241401110

241401110

Note: The vehicle-service scenario is an illustrative case study created for academic purposes. The organisation, figures and outcomes are fictional and are used to explain common low-code adoption challenges.

# Table of Contents

1. Abstract

2. Introduction

3. Background: What Is Low-Code?

4. Case Scenario: Vehicle Service Request Modernisation

5. Limitations Encountered

6. Impact Analysis

7. Comparison: Low-Code vs. Traditional Development

8. Lessons Learned and Recommendations

9. When Low-Code Is Still the Right Choice

10. Conclusion

11. References

241401110

# 1. Abstract

Low-code development platforms provide visual modelling, prebuilt components and configurable workflows that can speed up the delivery of business applications. They are useful for forms, dashboards and routine approvals. This case study considers an illustrative vehicle service request management system that handles customer requests, vehicle records, technician assignment, repair updates, notifications and reporting across multiple service centres. As transaction volumes and business rules grow, constraints in performance, integration, customisation, governance, security and long-term cost can become significant. The report analyses these challenges and recommends a hybrid architecture that uses low-code for suitable user-facing workflows while keeping performance-critical and complex functions in maintainable services.

# 2. Introduction

Vehicle service organisations increasingly need a coordinated digital system to receive service requests, track repair progress, manage technician workloads and keep customers informed. A low-code platform may appear attractive because it can deliver a working request form and dashboard quickly. However, a production system spanning multiple workshops may also require reliable integration with inventory, billing, vehicle history, customer messaging and external systems. It must preserve accurate records, apply role-based access and remain responsive during peak demand. This report assesses how low-code limitations may affect such a system when the initial prototype expands into a complex enterprise application.

# 3. Background: What Is Low-Code?

A low-code platform provides a visual development environment in which applications are assembled using drag-and-drop interfaces, model-driven logic, configurable connectors and reusable templates. Custom code can usually be added for exceptions, while the runtime, data layer and deployment pipeline are largely managed by the vendor. Strengths include rapid prototyping, built-in authentication options, simplified deployment and reduced dependence on specialised programmers. These same abstractions can limit control over performance, deep customisation and portability as system complexity increases.

241401110

# 4. Case Scenario: Vehicle Service Request Modernisation

## 4.1 Organisation and Objective

For this illustrative scenario, **MetroFleet Service Network** is a fictional organisation operating service centres in several cities. It wants to modernise service booking and repair tracking through a single system used by customers, service advisors, technicians and administrators. The objective is to digitise request intake, vehicle identification, priority handling, technician assignment, repair status, parts coordination and customer notifications.

## 4.2 Complexity Factors

| Factor | Illustrative requirement |
|---|---|
| Volume | Peak periods after holidays or fleet maintenance cycles may create large bursts of service requests. |
| Business rules | Warranty checks, service packages, vehicle types, priority levels and workshop-specific procedures. |
| Integration | Vehicle records, parts inventory, billing, payment gateways, SMS/email services and fleet systems. |
| Security and audit | Customers should see their own requests; technicians update assigned jobs; managers allocate work; admins manage roles. |
| Lifespan | The system is expected to evolve as service centres, vehicle types and business policies change. |

## 4.3 Project Outcome

The first version, containing a customer request form and a basic dashboard, is delivered quickly. During expansion, the team encounters slower responses during peak periods, increasingly complicated visual rules, and custom work needed to connect inventory and billing. Additional extensions are required for specialised workshop workflows. The organisation therefore considers a hybrid design: low-code remains responsible for selected forms and workflow screens, while core scheduling, integration and complex business rules move to independently tested services exposed through APIs.

241401110

# 5. Limitations Encountered

## 5.1 Scalability and Performance

A platform that performs well with a small pilot may struggle when several service centres submit and update requests at once. Generated queries, limited tuning options and vendor-dependent scaling can make it difficult to control response times. Delays in request lookup or status updates can disrupt service-desk operations and reduce customer confidence.

## 5.2 Handling Complex Business Logic

Rules for warranties, vehicle categories, service packages, parts availability, technician skills and appointment priorities can produce large visual workflows. Deeply nested conditions become difficult to review, test and maintain. Splitting complex rules into documented, unit-tested services can improve clarity and reduce the risk of inconsistent decisions.

## 5.3 Integration with Legacy and Third-Party Systems

Prebuilt connectors may support common services, but workshop-specific inventory, billing or fleet systems may require custom adapters. Differences in data formats, authentication, batch processing and retry behaviour can create fragile integrations. Integration services and well-defined APIs help isolate these dependencies.

## 5.4 Vendor Lock-In

Application models, workflow definitions and runtime behaviour may be specific to one vendor. If data structures or business rules cannot be exported in a portable form, changing platforms can require substantial redevelopment. Maintaining independent documentation, exportable data and API-based boundaries reduces the cost of a future transition.

## 5.5 Customisation and Flexibility Constraints

Specialised scheduling, advanced workload balancing, detailed audit events or unusual user-interface behaviour may exceed the platform's standard components. Workarounds can add complexity and technical debt, reducing the initial speed advantage.

241401110

## 5.6 Security, Compliance and Data Residency

The system stores customer contact details, vehicle information and service histories. The organisation must verify access controls, encryption, backup practices, audit logs and hosting locations. Misconfigured permissions can expose customer records or allow unauthorised changes to repair status, so access reviews and security testing are essential.

## 5.7 Governance and Application Sprawl

If individual service centres create separate apps without shared standards, duplicate vehicle records, inconsistent request categories and conflicting access rules may appear. Central ownership, approved data models, naming conventions and change control help keep the application portfolio consistent.

## 5.8 Cost at Scale

Licensing based on users, applications or resource consumption can increase as more workshops, technicians and customers join the system. Costs for consultants, custom integrations, extra environments and workarounds should be included in a multi-year total-cost-of-ownership estimate.

## 5.9 Skills, Testing and DevOps Maturity

Complex work still requires professional engineering skills. The team needs version control, automated regression testing, deployment approvals, monitoring and rollback procedures. Where the platform limits these practices, the delivery process should be supplemented with external testing and release controls.

## 5.10 Summary of Limitations

| Limitation | Observed symptom in this case | Severity |
|---|---|---|
| Scalability | Slow request search and updates during peak periods | High |
| Complex logic | Large visual workflows are difficult to test | High |
| Integration | Custom adapters for inventory and billing | High |
| Vendor lock-in | Costly migration and limited portability | High |
| Customisation | Workarounds for specialist workshop needs | Medium |
| Security/compliance | Access and audit controls need careful validation | High |
| Governance | Different centres create inconsistent processes | Medium |
| Cost at scale | Licensing and integration expenses grow | High |
| Skills/DevOps | Automated testing and release controls need extra effort | Medium |

241401110

## 6. Impact Analysis

The following table is an illustrative comparison of an initial project target and a possible outcome after complexity increases. The values are examples for analysis, not measurements from a real organisation.

| Metric | Initial target | Illustrative outcome |
|---|---|---|
| Delivery schedule | Initial request portal in 4 months | Core modernisation takes longer after redesign |
| Cost | Lower initial build cost | Additional licensing, integration and specialist work |
| Peak request handling | Responsive under expected peak load | Performance tuning or service separation required |
| Low-code coverage | All features planned on one platform | Low-code retained for selected forms and workflows |
| Custom code | Minimal extensions expected | Separate services added for complex rules and integrations |
| Operational effect | Faster booking and tracking | Potential delays and reduced confidence during transition |

Beyond direct cost and delivery time, poor performance or inconsistent request status can cause repeat calls, technician scheduling problems and lower customer satisfaction. These risks can be reduced by testing realistic workloads and integration failures before the system is expanded.

## 7. Comparison: Low-Code vs. Traditional Development

| Aspect | Low-code platform | Traditional development |
|---|---|---|
| Initial speed | Fast for forms and standard workflows | Usually slower at the start |
| Complex logic | Visual flows may become difficult to maintain | Flexible, testable code and service boundaries |
| Scalability control | Depends partly on platform runtime and plan | Greater architectural control |
| Integration depth | Convenient for supported connectors | Custom integration can be designed as needed |
| Portability | Potential vendor lock-in | Can use open standards and portable components |
| Cost profile | Lower initial effort; recurring licence costs may rise | Higher initial effort; costs depend on architecture and team |
| Skills | Platform-specific skills plus engineering | General software engineering skills |
| Best fit | Request forms, approvals, dashboards | Complex scheduling, high-load processing and core integrations |

## 8. Lessons Learned and Recommendations

- **Assess fit before committing.** Test peak request volumes, complicated service rules and the hardest integrations in a proof of concept.
- **Adopt a hybrid architecture.** Use low-code for suitable request forms, dashboards and workflow steps; keep performance-critical logic in independently tested services.
- **Design for exit.** Keep data exportable, document business rules outside the platform and define API boundaries.
- **Estimate total cost of ownership.** Include licences, environments, support, integration maintenance and specialist skills over several years.
- **Establish governance early.** Define common request categories, vehicle data standards, role permissions and clear ownership.

- **Retain engineering practices.** Use version control, automated tests, code reviews, monitoring and controlled deployments wherever possible.
- **Verify security.** Test role-based access, audit trails, backups and data handling before production release.

## 9. When Low-Code Is Still the Right Choice

Low-code remains useful for internal approval flows, service request forms, simple dashboards, proof-of-concept applications and routine notifications. It can help a service organisation validate a workflow quickly and involve operational staff in design. The important decision is to match the platform to the workload rather than assume it must host every component of the enterprise system.

241401110

# 10. Illustrative System Screenshots

The following project interface mock-ups are included to show how the vehicle service request management system could present request status, service intake, progress tracking and role permissions. They are illustrative screens created for this report, not captures from a live application.

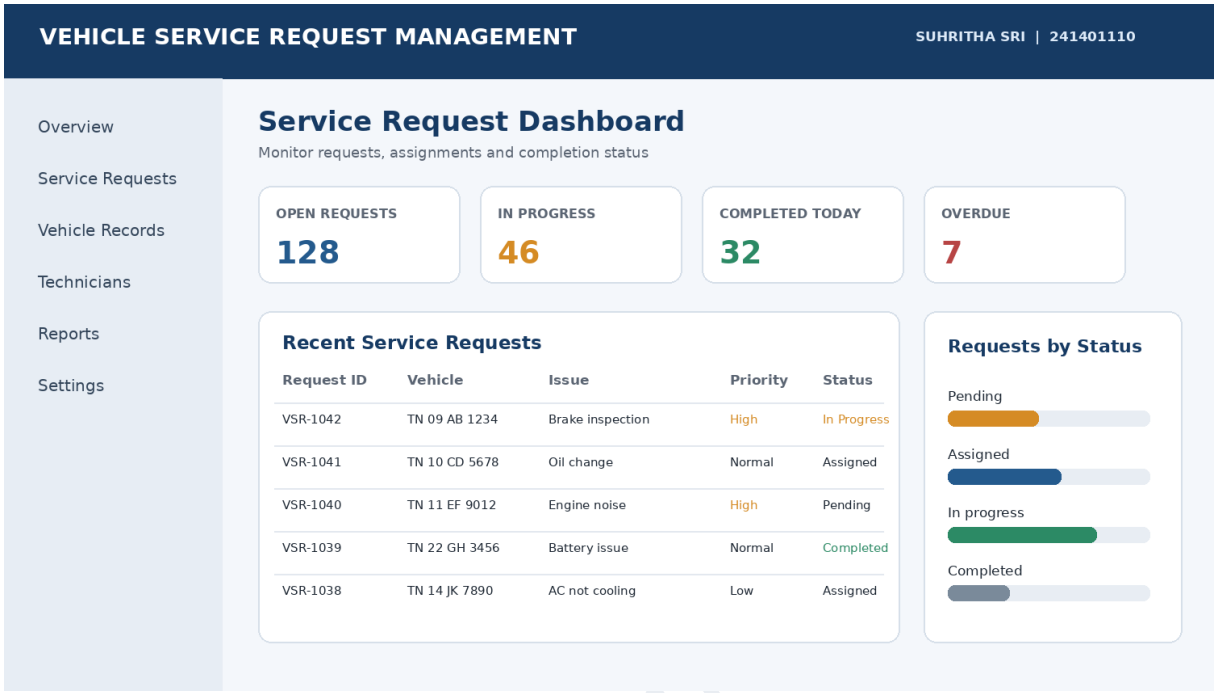



*Figure 1. Service request dashboard with open, in-progress, completed and overdue requests.*

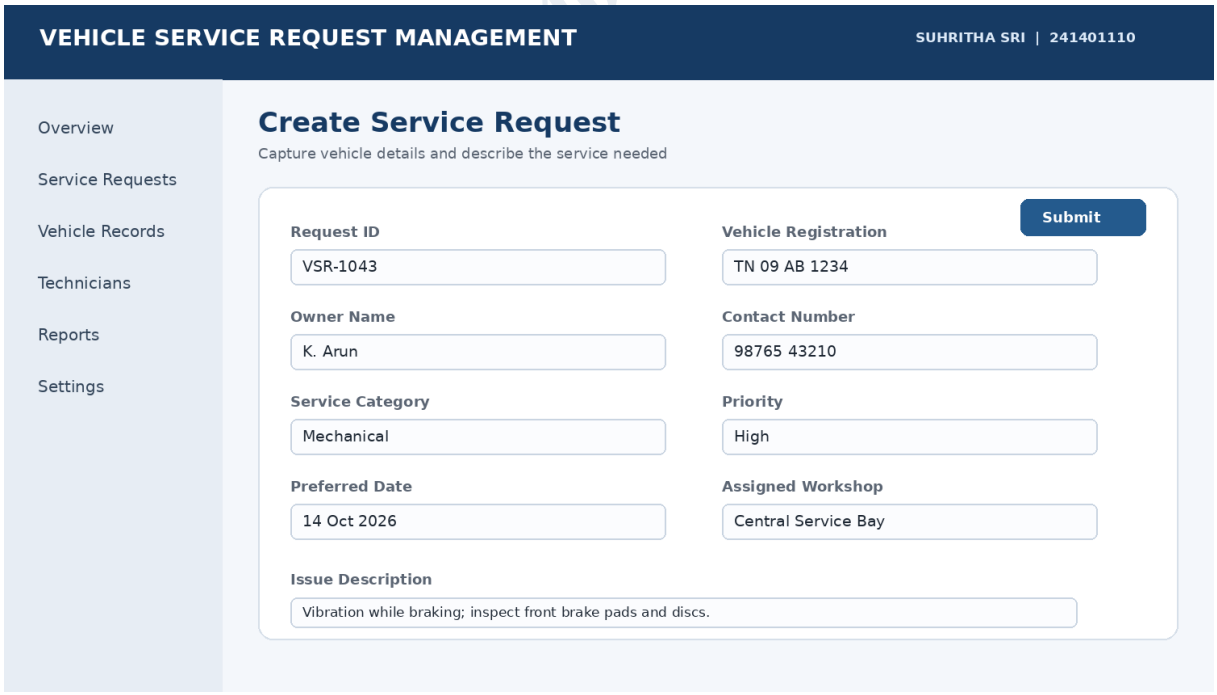



*Figure 2. Service request form for vehicle details, priority, service category and issue description.*

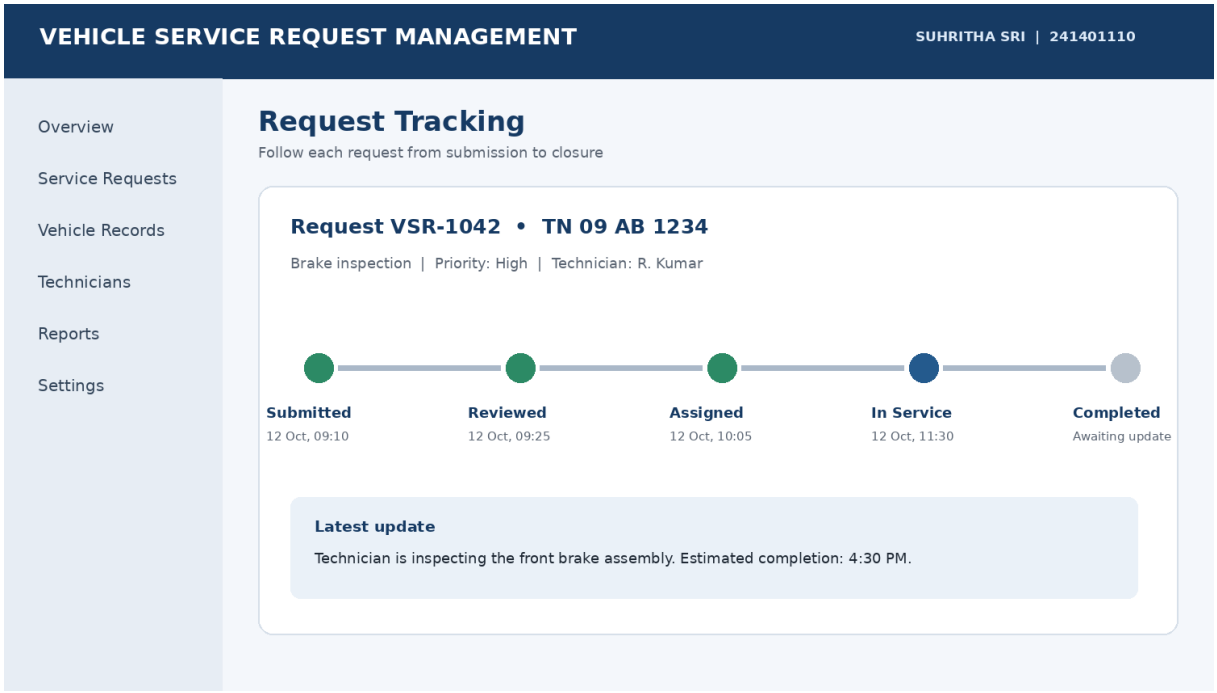



*Figure 3. Request tracking view showing stages from submission through completion.*

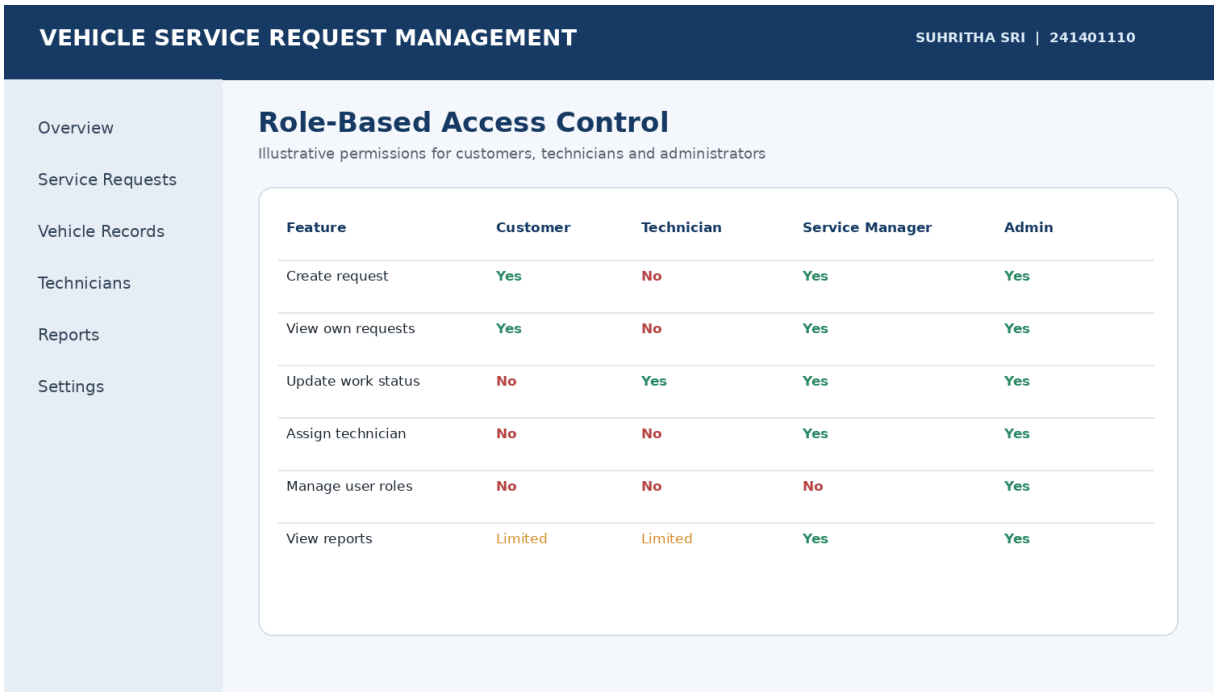



*Figure 4. Role-based access matrix for customers, technicians, service managers and administrators.*

## 11. Conclusion

Low-code platforms can accelerate delivery of vehicle service request forms, dashboards and standard workflows. However, the same platform may become restrictive when the system must handle peak request volumes, intricate service rules, multiple integrations, strong audit controls and long-term evolution. The illustrative case suggests a deliberate hybrid approach: use low-code where it offers clear speed and usability benefits, and use conventional services where control, performance, testability and portability are essential. Early governance, security validation and realistic cost analysis are necessary for a dependable enterprise solution.

## 12. References

• Bock, A. C., and Frank, U. (2021). “Low-Code Platform.” Business & Information Systems Engineering, 63(6), 733–740.

• Sahay, A., Indamutsa, A., Di Ruscio, D., and Pierantonio, A. (2020). “Supporting the Understanding and Comparison of Low-Code Development Platforms.” 46th Euromicro Conference on Software Engineering and Advanced Applications.

• Gartner, Inc. Research publications on enterprise low-code application platforms.

• Official product documentation for OutSystems, Mendix, Microsoft Power Apps and Appian; consulted as general background for low-code capabilities.

241401110